PROJECT REPORT

ON

ADVANCED INTERNET PROGRAMMING

PROJECT TOPIC

STUDENT MANAGEMENT SYSTEM

Submitted By:

Name: Anshu

UID: 26MCD10027

Section: 26MCD-1

Group:A

SubmittedTo:

Ms. Tanisha Rani

(E20679)

University Institute of Computing

ChandigarhUniversity,Gharuan,Mohali

BONAFIDE CERTIFICATE

This is to certify that the project report entitled **"Student Management System"** is the bonafide work carried out by **Anshu (UID: 26MCD10027)** under the guidance and supervision of **Ms. Tanisha Rani**, submitted in partial fulfilment of the academic requirements of the Master of Computer Applications (MCA) programme.

The project demonstrates the implementation of student record management using Java, JSP, Servlets, JDBC, Hibernate, and MySQL, with Apache Tomcat 11 as the web server.

Project Supervisor	**Head of the Department**
Ms. Tanisha Rani	Dr. Amanpreet Kaur Sandhu
(E20679)	(MCA)

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project supervisor, **Ms. Tanisha Rani**, for her invaluable guidance, constructive suggestions, and continuous support throughout the development of my project entitled **"Student Management System."** Her encouragement and guidance have been instrumental in the successful completion of this project.

I am deeply grateful to **Dr. Amanpreet Kaur Sandhu, Head of the Department (MCA)**, for providing the necessary academic support, facilities, and encouragement during the course of this project.

I would also like to extend my sincere thanks to all the faculty members for their valuable assistance and guidance. Finally, I am thankful to my family and friends for their constant encouragement and support.

I gratefully acknowledge everyone who contributed, directly or indirectly, to the successful completion of this project.

Anshu
26MCD10027

TABLE OF CONTENTS

S. No.	Contents	Page No
1	Introduction	5
2	Objectives	5
3	Problem Statement	5
4	Scope of the Project	5
5	System Architecture	6
6	Database Design	6
7	Project Implementation and Output	6-13
8	Working of the System	13
9	Testing and Results	13
10	Conclusion	13-14
11	Future Scope	14
12	References	14

1. INTRODUCTION

The Student Management System is a web-based application developed using Java, HTML, JDBC, Hibernate, and MySQL. It is designed to store and manage student information in an organized way.

In educational institutions, student information needs to be stored, updated, and retrieved regularly. Managing these records manually can be time-consuming. This project provides a simple digital solution for maintaining student records.

The application allows users to add student details, view existing records, update information, and delete records when required.

HTML is used to create the user interface. Java handles the application logic. JDBC connects Java to the MySQL database, while Hibernate provides object-relational mapping to manage database records through Java objects.

Apache Tomcat 11 is used to run the web application, and Apache NetBeans is used for development.

2. OBJECTIVES

1. To develop a simple web-based student management application.
2. To store student information in a MySQL database.
3. To establish database connectivity using JDBC.
4. To understand the use of Hibernate ORM.
5. To implement insert, view, update, and delete operations.
6. To provide a simple interface for entering student details.
7. To understand the communication between a web application and a database.
8. To gain practical knowledge of Java web application development.

3. PROBLEM STATEMENT

Managing student information manually can create several difficulties. Paper records require physical storage and may be difficult to search or update. Maintaining large numbers of records manually also requires additional time and effort.

The Student Management System aims to provide a simple application for managing student records digitally. It stores student information in a database and provides operations to add, view, update, and delete records.

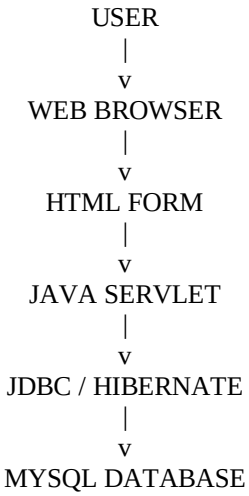
The project demonstrates how Java web technologies and database connectivity can be used to solve a basic record management problem.

4. SCOPE OF THE PROJECT

The application supports the following functions:

- **Add Student:** Enter and save a student's name and course.
- **View Students:** Display student records stored in the database.
- **Update Student:** Modify the name or course of an existing student.
- **Delete Student:** Remove a student record from the database.

5. SYSTEM ARCHITECTURE



6. DATABASE DESIGN (SQL Code)

```
USE aip_studentdb;

CREATE TABLE IF NOT EXISTS students (
   id INT PRIMARY KEY AUTO_INCREMENT,
   name VARCHAR(50) NOT NULL,
   course VARCHAR(50) NOT NULL
);
```

7. PROJECT IMPLEMENTATION

7.1 Project Structure

```
StudentManagementAIP
├── pom.xml
└── src
    └── main
        ├── java
        │   └── com
        │       └── example
        │           ├── Student.java
        │           ├── HibernateUtil.java
        │           └── StudentServlet.java
        ├── resources
        │   └── hibernate.cfg.xml
        └── webapp
            └── index.html
```

7.2 Maven Configuration (File: pom.xml)

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
```

```
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
        https://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>

    <groupId>com.example</groupId>
    <artifactId>StudentManagementAIP</artifactId>
    <version>1.0</version>
    <packaging>war</packaging>

    <properties>
       <maven.compiler.release>17</maven.compiler.release>
       <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    </properties>

    <dependencies>
       <dependency>
          <groupId>jakarta.servlet</groupId>
          <artifactId>jakarta.servlet-api</artifactId>
          <version>6.1.0</version>
          <scope>provided</scope>
       </dependency>

       <dependency>
          <groupId>org.hibernate.orm</groupId>
          <artifactId>hibernate-core</artifactId>
          <version>6.6.13.Final</version>
       </dependency>

       <dependency>
          <groupId>com.mysql</groupId>
          <artifactId>mysql-connector-j</artifactId>
          <version>9.4.0</version>
       </dependency>
    </dependencies>

    <build>
       <finalName>StudentManagementAIP</finalName>
    </build>
</project>
```

7.3 Student Entity Class (File: Student.java)

```
package com.example;

import jakarta.persistence.*;

@Entity
@Table(name = "students")
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;
```

```
    private String name;
    private String course;

    public Student() {
    }

    public int getId() {
        return id;
    }

    public String getName() {
        return name;
    }

    public String getCourse() {
        return course;
    }

    public void setName(String name) {
        this.name = name;
    }

    public void setCourse(String course) {
        this.course = course;
    }
}
```

7.4 Hibernate Configuration (File: hibernate.cfg.xml)

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-configuration PUBLIC
 "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
 "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">

<hibernate-configuration>
    <session-factory>

        <property name="hibernate.connection.driver_class">
            com.mysql.cj.jdbc.Driver
        </property>

        <property name="hibernate.connection.url">
            jdbc:mysql://localhost:3306/aip_studentdb
        </property>

        <property name="hibernate.connection.username">root</property>
        <property name="hibernate.connection.password"></property>

        <property name="hibernate.dialect">
            org.hibernate.dialect.MySQLDialect
        </property>

        <mapping class="com.example.Student"/>

    </session-factory>
```

```
</hibernate-configuration>
```

7.5 Hibernate Utility Class (File: HibernateUtil.java)

```
package com.example;

import org.hibernate.SessionFactory;
import org.hibernate.cfg.Configuration;

public class HibernateUtil {

    private static final SessionFactory factory =
        new Configuration()
            .configure("hibernate.cfg.xml")
            .buildSessionFactory();

    public static SessionFactory getSessionFactory() {
        return factory;
    }
}
```

7.6 HTML Student Form (File: index.html)

```
<!DOCTYPE html>
<html>
<head>
    <title>Student Management System</title>
</head>
<body>

    <h2>Student Management System</h2>

    <form action="students" method="post">
        <input type="hidden" name="action" value="add">

        Student Name:
        <input type="text" name="name" required>
        <br><br>

        Course:
        <input type="text" name="course" required>
        <br><br>

        <button type="submit">Add Student</button>
    </form>

    <br>
    <a href="students">View All Students</a>

</body>
</html>
```

7.7 Java Servlet (File: StudentServlet.java)

```
package com.example;

import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
```

```
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

import java.io.IOException;
import java.io.PrintWriter;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;

import org.hibernate.Session;
import org.hibernate.Transaction;

@WebServlet("/students")
public class StudentServlet extends HttpServlet {

   @Override
   protected void doPost(HttpServletRequest request,
                         HttpServletResponse response)
           throws ServletException, IOException {

       String action = request.getParameter("action");

try {
   Class.forName("com.mysql.cj.jdbc.Driver");

   if ("add".equals(action)) {
            String name = request.getParameter("name");
            String course = request.getParameter("course");

            try (Connection con = DriverManager.getConnection(
                  "jdbc:mysql://localhost:3306/aip_studentdb",
                  "root", "");
                PreparedStatement ps = con.prepareStatement(
                  "INSERT INTO students(name, course) VALUES (?, ?)")) {

               ps.setString(1, name);
               ps.setString(2, course);
               ps.executeUpdate();
            }

         } else if ("update".equals(action)) {

            int id = Integer.parseInt(request.getParameter("id"));

            try (Session session =
                  HibernateUtil.getSessionFactory().openSession()) {

               Transaction tx = session.beginTransaction();
               Student s = session.get(Student.class, id);

               if (s != null) {
                  s.setName(request.getParameter("name"));
                  s.setCourse(request.getParameter("course"));
               }
```

```
            tx.commit();
        }

    } else if ("delete".equals(action)) {

        int id = Integer.parseInt(request.getParameter("id"));

        try (Session session =
                HibernateUtil.getSessionFactory().openSession()) {

            Transaction tx = session.beginTransaction();
            Student s = session.get(Student.class, id);

            if (s != null) {
                session.remove(s);
            }

            tx.commit();
        }
    }

    response.sendRedirect(request.getContextPath() + "/students");

  } catch (Exception e) {
    throw new ServletException("Database operation failed.", e);
  }
}

@Override
protected void doGet(HttpServletRequest request,
                HttpServletResponse response)
    throws ServletException, IOException {

  response.setContentType("text/html;charset=UTF-8");

  try (PrintWriter out = response.getWriter();
       Session session =
           HibernateUtil.getSessionFactory().openSession()) {

    var students = session
        .createQuery("FROM Student ORDER BY id", Student.class)
        .list();

    out.println("<html><body>");
    out.println("<h2>Student Records</h2>");
    out.println("<a href='" + request.getContextPath()
            + "/'>Add New Student</a><br><br>");

    out.println("<table border='1' cellpadding='8'>");
    out.println("<tr><th>ID</th><th>Name</th>"
            + "<th>Course</th><th>Actions</th></tr>");

    for (Student s : students) {

        out.println("<tr><td>" + s.getId() + "</td>");
        out.println("<td>" + s.getName() + "</td>");
```

```
                out.println("<td>" + s.getCourse() + "</td><td>");

                out.println("<form action='"
                    + request.getContextPath()
                    + "/students' method='post'>");

                out.println("<input type='hidden' name='action' value='update'>");
                out.println("<input type='hidden' name='id' value='"
                    + s.getId() + "'>");
                out.println("<input name='name' value='"
                    + s.getName() + "' required>");
                out.println("<input name='course' value='"
                    + s.getCourse() + "' required>");
                out.println("<button type='submit'>Update</button>");
                out.println("</form>");

                out.println("<form action='"
                    + request.getContextPath()
                    + "/students' method='post'>");

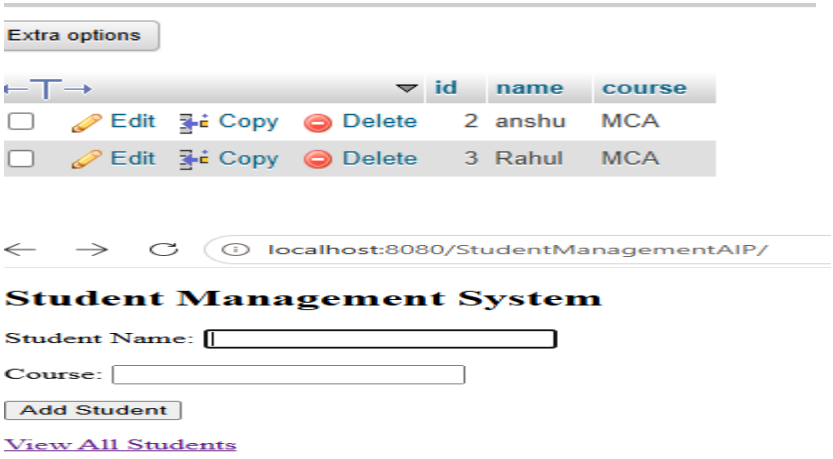
                out.println("<input type='hidden' name='action' value='delete'>");
                out.println("<input type='hidden' name='id' value='"
                    + s.getId() + "'>");
                out.println("<button type='submit'>Delete</button>");
                out.println("</form>");

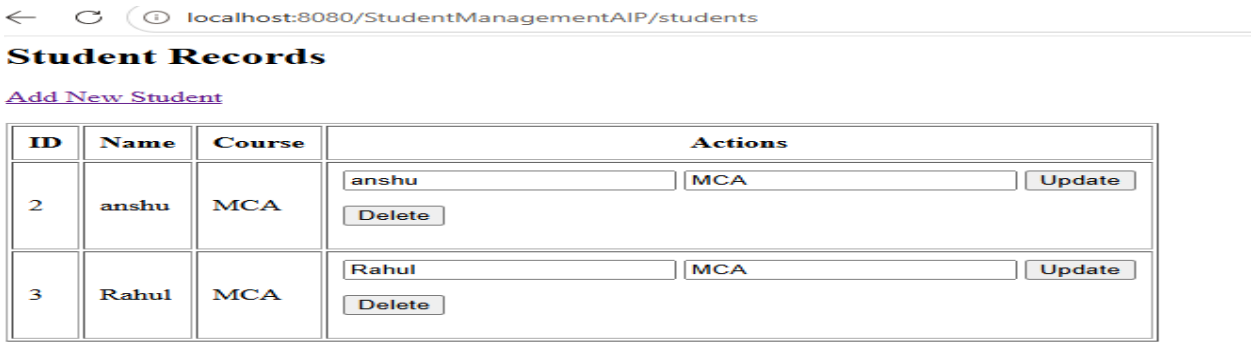
                out.println("</td></tr>");
            }

            out.println("</table></body></html>");

        } catch (Exception e) {
            throw new ServletException("Unable to display student records.", e);
        }
    }
}
```

Output





8. WORKING OF THE SYSTEM

1. The user opens the Student Management System in a web browser.
2. The user enters a student name and course.
3. The HTML form sends the information to the Java Servlet.
4. The Servlet reads the form data.
5. JDBC inserts new records into MySQL.
6. Hibernate retrieves, updates, and deletes student records.
7. The application displays the available records in a table.

9. TESTING AND RESULTS

Test Case	**Expected Result**
Enter valid student details	The record should be inserted
Open the student list	Saved records should be displayed
Modify student details	The record should be updated
Delete a student record	The selected record should be removed
Submit the form without required fields	The browser should request the missing information

Result

The project is designed to demonstrate student record management using JDBC and Hibernate. Its successful operation must be confirmed by building and running the application and testing each operation.

10. CONCLUSION

The Student Management System demonstrates the practical use of Advanced Internet Programming concepts. It combines HTML, Java, JDBC, Hibernate, and MySQL to manage student information.

The application provides basic functions for adding, viewing, updating, and deleting student records. JDBC provides database connectivity, while Hibernate simplifies record management through Java objects.
This project helped me understand the structure of a Java web application and the communication between the client, server, and database.

11. FUTURE SCOPE

1. User login and authentication.
2. Student attendance management.
3. Student marks and result management.
4. Search and filter options.
5. Data validation and improved error messages.
6. Report generation.
7. A more attractive and responsive user interface.

12. REFERENCES

1. Java Documentation.
2. JDBC Documentation.
3. Hibernate ORM Documentation.
4. MySQL Reference Manual.
5. Apache Tomcat Documentation.
6. HTML Documentation.